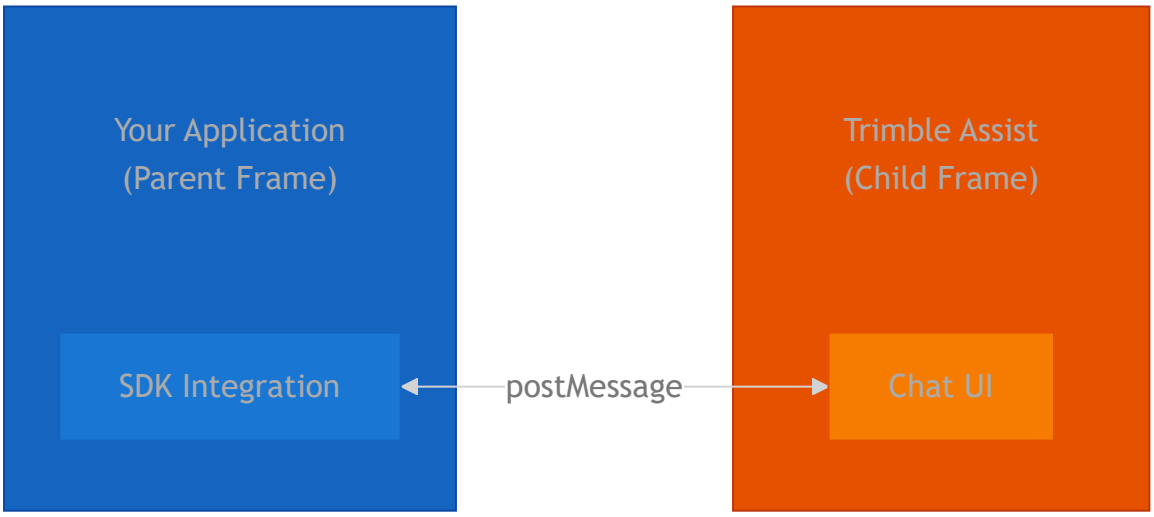
Iframe integration

This guide covers the direct iFrame integration path for embedding the Trimble Assist UI in your app.

Use this approach when you need lower-level control. If you are using React or Angular, start with Framework Components, which is the recommended path for most teams.

The Agentic Platform Iframe SDK enables seamless bi-directional communication between your host application and the embedded Trimble Assist Chat UI. Using the browser's `postMessage` API, your application can:

- Provide configuration and authentication to the Chat UI
- Register local tools that the AI agent can invoke
- Inject contextual information into agent conversations
- Listen and respond to UI events
- Dynamically update the Chat UI configuration



Installation

The SDK is hosted on Trimble's private JFrog Artifactory. Before installing, you must configure your project to authenticate with the registry.

1. **Create an `.npmrc` file:** Create a file named `.npmrc` in the root of your project and add the following configuration:

```
# Default registry
registry=https://registry.npmjs.org/

# Trimble external packages
@trimble-agentic-external-npm-
local:registry=https://artifactory.trimble.tools/artifactory/api/npm/t
agentic-external-npm-local/
//artifactory.trimble.tools/artifactory/api/npm/trimble-agentic-extern
local/:_authToken=YOUR_AUTH_TOKEN
```

2. **Generate an Identity Token:**
   - Log in to Trimble Artifactory.
   - Navigate to your User Profile.
   - Generate a new **Identity Token**.
3. **Add the Token:** Replace `YOUR_AUTH_TOKEN` in your `.npmrc` file with the generated token.

> **DANGER**
>
> **NEVER commit your `.npmrc` file if it contains your personal auth token!** Make sure to add `.npmrc` to your project's `.gitignore`.

4. **Install the SDK:** Now you can install the package using npm, yarn, or pnpm:

```
npm install @trimble-agentic-external-npm-local/agentic-platform-
sdk-iframe-typescript@1.0.1-rc.7
```

Quick Start

Here's a minimal example to get the Chat UI running in your application:

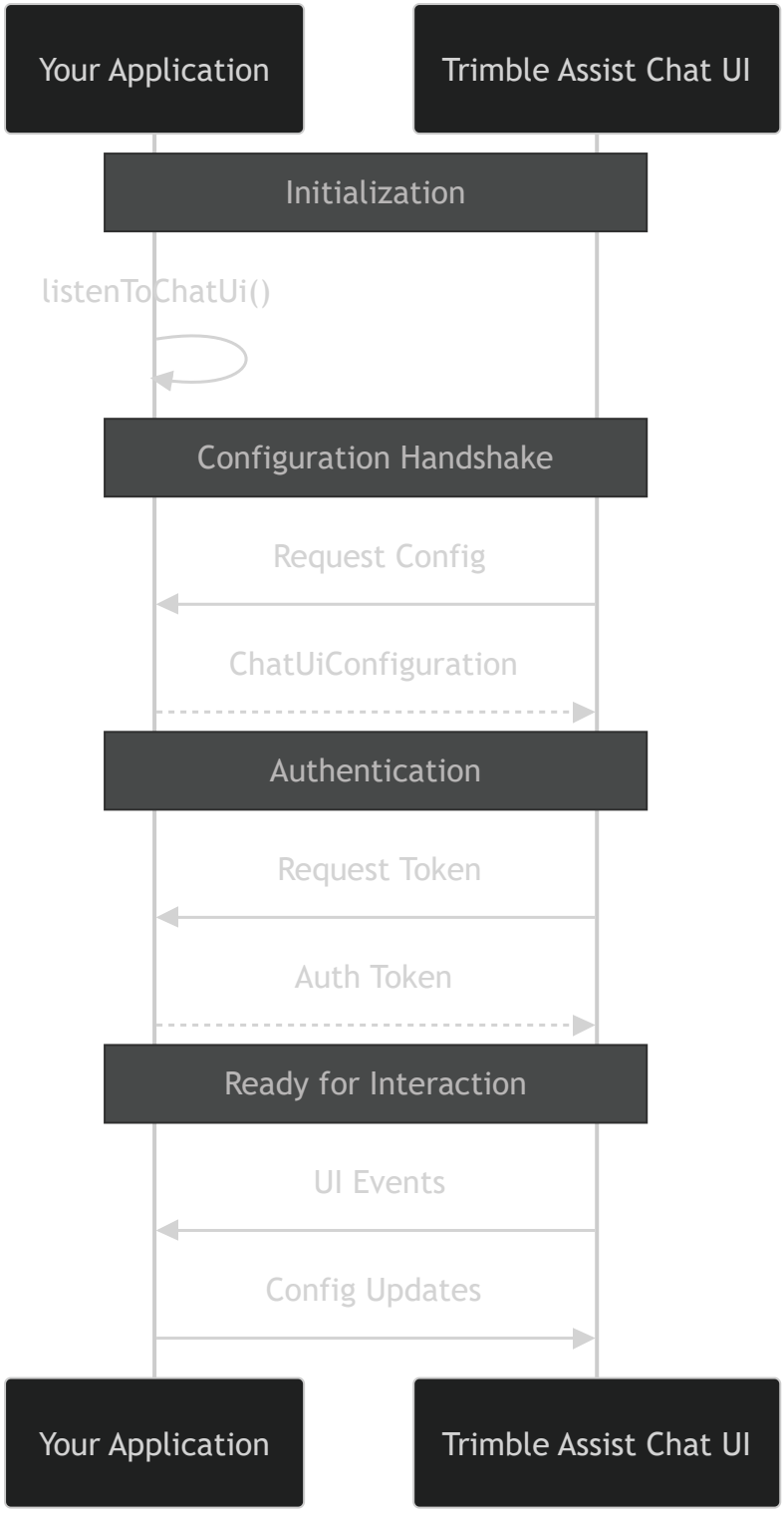
```
import {
  listenToChatUi,
  CHAT_UI_URLS,
```

```
  type ChatUiConfiguration,
  ContentVariants,
  ChatUiVariants,
} from "@trimble-agentic-external-npm-local/agentic-platform-sdk-
iframe-typescript";

// 1. Set the environment and get the Chat UI URL
const environment = "prod";
const chatUiUrl = CHAT_UI_URLS[environment];

// 2. Create and embed the iframe
const iframe = document.createElement("iframe");
iframe.src = chatUiUrl;
iframe.style.width = "100%";
iframe.style.height = "600px";
document.body.appendChild(iframe);

// 3. Define your configuration provider
const getConfig = (): ChatUiConfiguration => ({
  environment,
  agentId: "your-agent-id",
  uiConfig: {
    theme: "light",
    contentVariant: ContentVariants.Chat,
    variant: ChatUiVariants.Full,
    chatInput: {
      buttons: [],
      hideModelSelection: true,
    },
  },
  localization: {
    locale: "en",
  },
});

// 4. Define your token provider
const getToken = async (): Promise<string> => {
  // Return the access token from your Trimble Identity (TID)
integration
  return "your-tid-access-token";
};

// 5. Start listening for Chat UI requests
const cleanup = listenToChatUi(
  iframe,
  chatUiUrl, // Use the same URL as target origin
  getConfig,
```

```
  getToken,
);

// 6. Clean up when done (e.g., on component unmount)
// cleanup();
```

Communication Flow

Understanding how messages flow between your application and the Chat UI is key to effective integration.



Core API Reference

`listenToChatUi`

The primary function for establishing communication with the Chat UI. It handles all incoming requests and dispatches appropriate responses.

```
function listenToChatUi(
  iframe: HTMLIFrameElement,
  targetOrigin: string,
  provideChatUiConfig: () => ChatUiConfiguration,
  provideChatUiToken: () => Promise<string>,
  onBeforeRun?: OnBeforeRunConfig | OnBeforeRunProvider,
  onUnauthorized?: () => Promise<string | undefined>,
): () => void;
```

Parameters

Parameter	Type	Description
`iframe`	`HTMLIFrameElement`	The iframe element containing the Chat UI
`targetOrigin`	`string`	The origin URL of the Chat UI for security validation
`provideChatUiConfig`	`() => ChatUiConfiguration`	Function that returns the current configuration
`provideChatUiToken`	`() => Promise<string>`	Async function that returns the current auth token
`onBeforeRun`	`OnBeforeRunConfig | OnBeforeRunProvider`	Optional static config or async provider for tools and context
`onUnauthorized`	`() => Promise<string | undefined>`	Optional handler for unauthorized flow (returns optional message)

Returns

A cleanup function that removes the message event listener.

Example

```
import {
  listenToChatUi,
  CHAT_UI_URLS,
  type ChatUiConfiguration,
  type OnBeforeRunConfig,
  ContentVariants,
} from "@trimble-agentic-external-npm-local/agentic-platform-sdk-iframe-typescript";

const iframe = document.getElementById("chat-iframe") as HTMLIFrameElement;
const CHAT_UI_ORIGIN = CHAT_UI_URLS["prod"];

const onBeforeRun: OnBeforeRunConfig = {
  tools: {
    runTime: {},
    global: {},
  },
  runContext: {
    context: [
      { description: "Current user", value: "john.doe@example.com" },
      { description: "Project ID", value: "proj-12345" },
    ],
  },
};

const stopListening = listenToChatUi(
  iframe,
  CHAT_UI_ORIGIN,
  () => getConfiguration(),
  () => getAuthToken(),
  onBeforeRun,
);
```

updateConfig

Dynamically update the Chat UI configuration after initial setup. Use this when your application state changes and the Chat UI needs to reflect those changes.

```
function updateConfig(
  iframe: HTMLIFrameElement,
  targetOrigin: string,
```

```
  config: ChatUiConfiguration,
): void;
```

Example

```
import { updateConfig } from "@trimble-agentic-external-npm-
local/agentic-platform-sdk-iframe-typescript";

// Toggle theme based on user preference
function toggleTheme(isDark: boolean) {
  updateConfig(iframe, CHAT_UI_ORIGIN, {
    ...currentConfig,
    uiConfig: {
      ...currentConfig.uiConfig,
      theme: isDark ? "dark" : "light",
    },
  });
}
```

`listenToChatUiEvents`

Subscribe to UI events emitted by the Chat UI. This allows your application to react to user interactions within the iframe.

```
function listenToChatUiEvents(
  targetOrigin: string,
  onEvent: (event: ChatUiEvent) => void,
): () => void;
```

Example

```
import {
  listenToChatUiEvents,
  ChatUiEventTypes,
  type ChatUiEvent,
} from "@trimble-agentic-external-npm-local/agentic-platform-sdk-
iframe-typescript";

const stopListeningEvents = listenToChatUiEvents(
  CHAT_UI_ORIGIN,
  (event: ChatUiEvent) => {
```

```
    switch (event.type) {
      case ChatUiEventTypes.OnAgentSelect:
        console.log("User selected agent:", event.payload);
        break;
      case ChatUiEventTypes.OnNewChat:
        console.log("User started a new chat");
        break;
      case ChatUiEventTypes.OnSignIn:
        handleSignIn();
        break;
      case ChatUiEventTypes.OnChatInputButtonClick:
        handleCustomButtonClick(event.payload);
        break;
      case ChatUiEventTypes.OnClose:
        console.log("User closed the chat");
        hideChatPanel();
        break;
    }
  },
);
```

`subscribeToChatEvents`

Subscribe to internal Chat lifecycle events from the iframe. These are state change and lifecycle events useful for analytics, telemetry, and host application synchronization.

```
function subscribeToChatEvents(
  targetOrigin: string,
  onEvent: (event: ChatEvent) => void,
): () => void;
```

Parameters

Parameter	Type	Description
`targetOrigin`	`string`	The origin URL to validate incoming messages
`onEvent`	`(event: ChatEvent) => void`	Callback function to handle incoming events

Available Event Types

Event Type	Payload	Description
`run:started`	`{ runId, threadId, agentId }`	Agent run has started
`run:ended`	`{ runId, threadId, agentId, status: 'completed' | 'failed' | 'cancelled' }`	Agent run has ended
`thread:created`	`{ threadId }`	New conversation thread created
`thread:changed`	`{ threadId }`	User switched to different thread
`agent:changed`	`{ agentId }`	User selected an agent
`agentModel:changed`	`{ modelId }`	User changed the model

Example

```
import {
  subscribeToChatEvents,
  type ChatEvent,
} from "@trimble-agentic-external-npm-local/agentic-platform-sdk-iframe-typescript";

const unsubscribe = subscribeToChatEvents(
  CHAT_UI_ORIGIN,
  (event: ChatEvent) => {
    switch (event.type) {
      case "run:started":
        analytics.track("run_started", {
          runId: event.payload.runId,
          agentId: event.payload.agentId,
        });
        break;
      case "run:ended":
```

```
        analytics.track("run_ended", {
          runId: event.payload.runId,
          status: event.payload.status,
        });
        break;
      case "thread:created":
        console.log("New thread:", event.payload.threadId);
        break;
      case "agent:changed":
        updateHostAppState({ selectedAgent: event.payload.agentId });
        break;
    }
  },
);
```

Configuration

ChatUiConfiguration

The main configuration object that controls the Chat UI behavior and appearance.

```
type ChatUiConfiguration = {
  environment: "development" | "stage" | "prod";
  onBeforeRunTimeout?: number;
  uiConfig: UiConfig;
  localization: Localization;
  agentId: string;
  threadId?: string;
};
```

UiConfig

Controls the visual appearance and UI behavior.

```
type UiConfig = {
  theme: "dark" | "light";
  contentVariant: ContentVariants;
  variant: ChatUiVariants;
  chatInput: {
    buttons: ChatInputButton[];
    hideModelSelection: boolean;
```

```
  };
  showSignIn?: boolean;
  productAgents?: string[];
};
```

Content Variants

Variant	Value	Description
`Chat`	`0`	Standard chat interface for conversations
`AgentCards`	`1`	Agents page for discovering and selecting agents

Chat UI Variants

Variant	Value	Description
`Minimal`	`'minimal'`	Minimal UI with essential chat features only
`Full`	`'full'`	Full-featured UI with all capabilities
`Narrow`	`'narrow'`	Narrow layout optimized for sidebar or panel embedding

```
import { ChatUiVariants } from "@trimble-agentic-external-npm-
local/agentic-platform-sdk-iframe-typescript";

// For sidebar embedding
const sidebarConfig = {
  ...baseConfig,
  uiConfig: {
    ...baseConfig.uiConfig,
    variant: ChatUiVariants.Narrow,
  },
};
```

Custom Input Buttons

Add custom action buttons to the chat input area:

```
const config: ChatUiConfiguration = {
  environment: "prod",
  agentId: "your-agent-id",
  uiConfig: {
    theme: "light",
    contentVariant: ContentVariants.Chat,
    variant: ChatUiVariants.Full,
    chatInput: {
      buttons: [
        { id: "attach-file", label: "Attach File" },
        { id: "use-template", label: "Templates", disabled: false },
      ],
      hideModelSelection: true,
    },
  },
  localization: {},
};
```

When users click these buttons, you'll receive an `OnChatInputButtonClick` event with the button's `id` as the payload.

UI Events

The Chat UI emits events that your application can listen to and respond accordingly.

Event Type	Payload	Description
`OnAgentSelect`	`string` (agentId)	User selected an agent
`OnThreadSelect`	`undefined`	User selected a conversation thread
`OnNewChat`	`undefined`	User started a new conversation
`OnExploreAgents`	`undefined`	User clicked to explore agents
`OnCreateAgent`	`undefined`	User clicked to create an agent
`OnSignIn`	`undefined`	User clicked sign in

Event Type	Payload	Description
`OnMyTrimbleClick`	`undefined`	User clicked My Trimble link
`OnChatInputButtonClick`	`string` (buttonId)	User clicked a custom input button
`OnClose`	`undefined`	User closed the Chat UI

Local Tools

Local tools allow you to extend the AI agent's capabilities with functionality from your host application. There are two types of tools:

Runtime Tools

Runtime tools are defined at conversation time and include a full tool definition. The AI agent sees these tools and can decide to invoke them.

```
import {
  type OnBeforeRunConfig,
  type Tool,
} from "@trimble-agentic-external-npm-local/agentic-platform-sdk-
iframe-typescript";

const getWeatherTool: Tool = {
  name: "get_current_weather",
  description: "Get the current weather for a specific location",
  parameters: {
    type: "object",
    properties: {
      location: {
        type: "string",
        description: 'City name, e.g., "San Francisco, CA"',
      },
      units: {
        type: "string",
        enum: ["celsius", "fahrenheit"],
        description: "Temperature units",
      },
    },
```

```
    required: ["location"],
  },
};

const onBeforeRun: OnBeforeRunConfig = {
  tools: {
    runTime: {
      get_current_weather: {
        definition: getWeatherTool,
        callback: async (args) => {
          const { location, units = "celsius" } = args as {
            location: string;
            units?: string;
          };

          // Call your weather API
          const weather = await fetchWeather(location, units);
          return JSON.stringify(weather);
        },
        timeOutInMs: 10000, // Optional timeout
      },
    },
    global: {},
  },
};
```

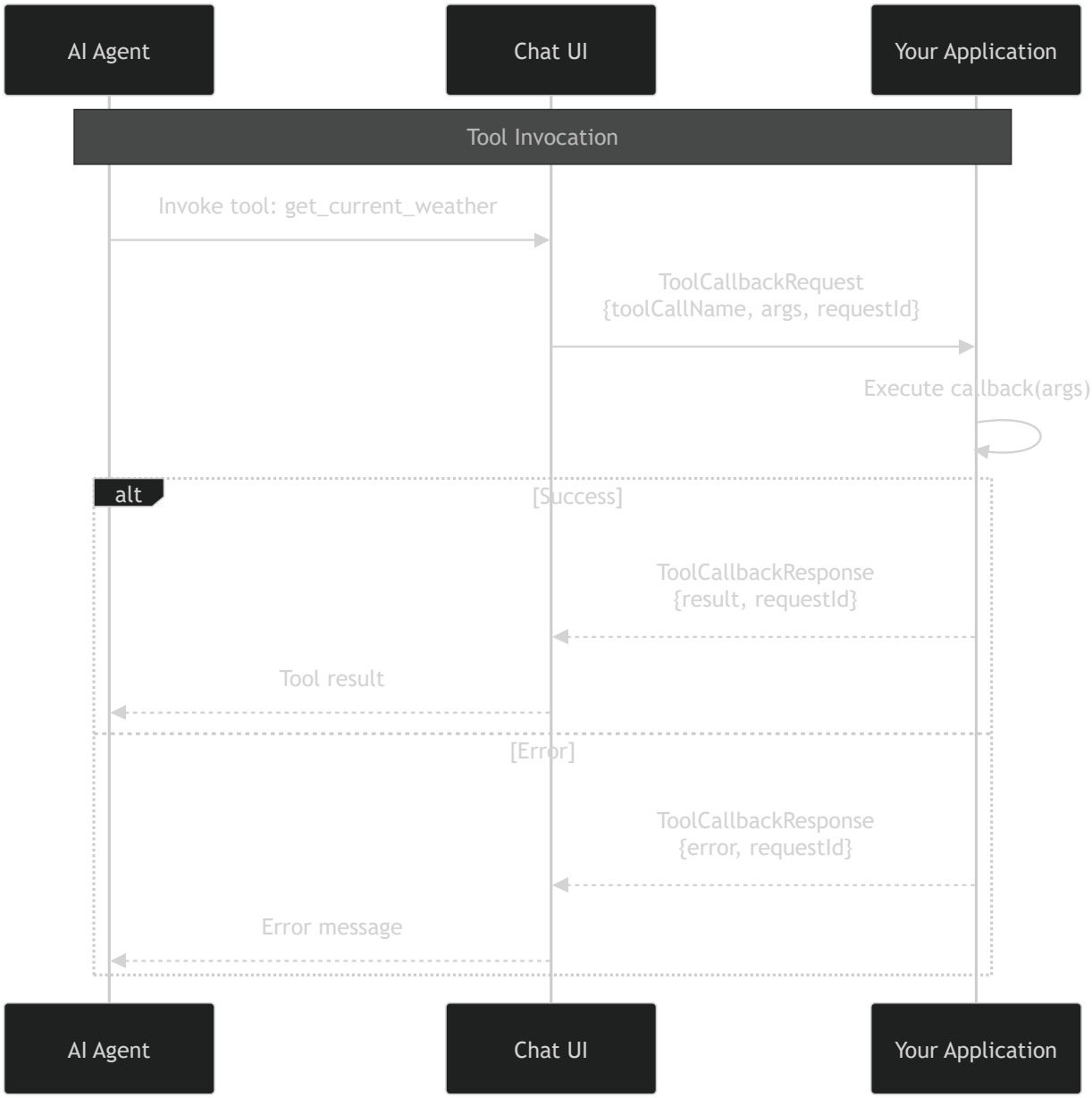
Global Tools

Global tools are callbacks for tools that are already registered in the agent definition but execute in your host application.

```
const onBeforeRun: OnBeforeRunConfig = {
  tools: {
    runTime: {},
    global: {
      // This tool is already defined in the agent, but executes
locally
      open_document: {
        callback: async (args) => {
          const { documentId } = args as { documentId: string };
          await openDocumentInApp(documentId);
          return "Document opened successfully";
        },
        timeOutInMs: 5000,
      },
```

```
    },
  },
};
```

Tool Execution Flow



Tool Definition Schema

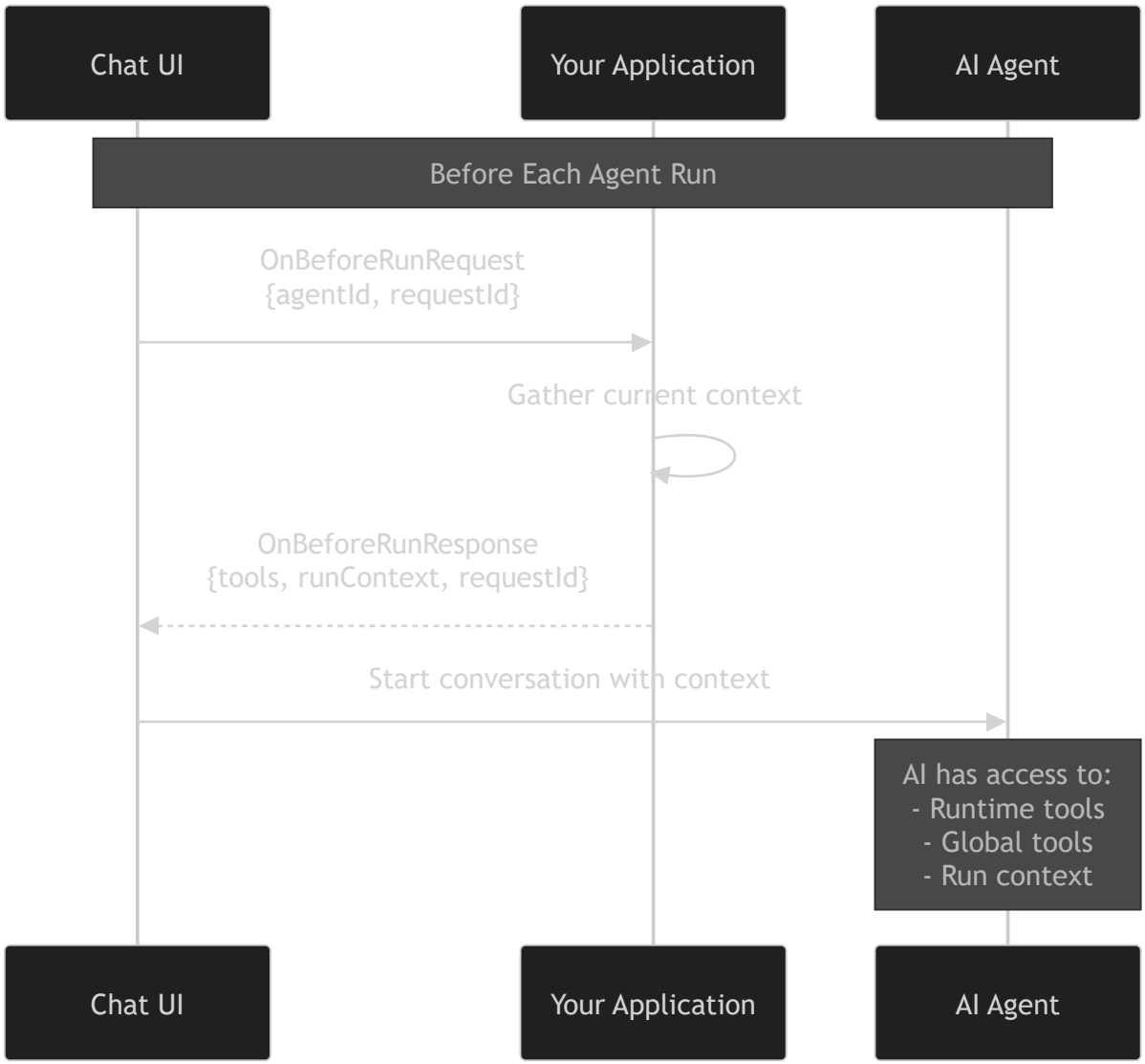
```
type Tool = {
  name: string; // Unique identifier for the tool
  description: string; // Description for the AI to understand when to
use it
  parameters?: Record<string, unknown>; // JSON Schema for parameters
};
```

Run Context

Inject contextual information into each agent conversation. This context helps the AI understand the current state of your application.

```
const onBeforeRun: OnBeforeRunConfig = {
  runContext: {
    context: [
      {
        description: "Currently logged in user",
        value: "jane.smith@company.com",
      },
      {
        description: "Active project",
        value: "Highway Extension Project - Phase 2",
      },
      {
        description: "User role",
        value: "Project Manager",
      },
      {
        description: "Current date",
        value: new Date().toISOString(),
      },
    ],
  },
};
```

Context Flow



Localization

Configure the Chat UI language using the `locale` property:

```
const config: ChatUiConfiguration = {
  environment: "prod",
  agentId: "your-agent-id",
  uiConfig: {
    theme: "light",
    contentVariant: ContentVariants.Chat,
    variant: ChatUiVariants.Full,
    chatInput: { buttons: [], hideModelSelection: true },
  },
  localization: {
    locale: "en", // BCP-47 locale tag (e.g., "en", "de", "fr", "es")
  },
};
```

DEPRECATED PROPERTIES

The following localization properties are **deprecated**:

- `selectedLanguage` — Use `locale` instead. The SDK will emit a console warning if you use this property.
- `translations` — Use the bundled Chat UI translations via `locale` instead. Custom translations are still accepted as overrides but are no longer the recommended approach.

```
// ❌ Deprecated approach
localization: {
  translations: { en: { chat: { newChat: "New" } } },
  selectedLanguage: "en",
}

// ✅ Recommended approach
localization: {
  locale: "en",
}
```

Supported Locales

The SDK exports `CHAT_UI_SUPPORTED_LOCALES` which contains the list of available locale codes:

```
import { CHAT_UI_SUPPORTED_LOCALES } from "@trimble-agentic-external-npm-local/agentic-platform-sdk-iframe-typescript";

// Check if a locale is supported
const isSupported = CHAT_UI_SUPPORTED_LOCALES.includes("de");
```

Localization Type Reference

```
type Localization = {
  locale?: string; // BCP-47 locale tag (preferred)
  selectedLanguage?: string; // @deprecated - use locale instead
  translations?: Record<string, string>; // @deprecated - use bundled
```

```
translations
};
```

Product Agents

Filter which agents are displayed in the Chat UI by specifying their IDs:

```
const config: ChatUiConfiguration = {
  environment: "prod",
  agentId: "default-agent-id",
  uiConfig: {
    theme: "light",
    contentVariant: ContentVariants.AgentCards,
    variant: ChatUiVariants.Full,
    chatInput: { buttons: [], hideModelSelection: true },
    productAgents: ["agent-uuid-1", "agent-uuid-2", "agent-uuid-3"],
  },
  localization: { locale: "en" },
};
```

When `productAgents` is set, only the specified agents will be shown in the agent selection UI. This is useful for embedding product-specific agent experiences.

Security Considerations

Origin Validation

Always specify the exact `targetOrigin` when communicating with the iframe:

```
// ✅ Correct - use the SDK-provided URLs
listenToChatUi(iframe, CHAT_UI_URLS['prod'], ...);

// ❌ Incorrect - never use wildcard in production
listenToChatUi(iframe, '*', ...);
```

Token Management

- Never log or expose authentication tokens

- The `provideChatUiToken` function is called each time a token is needed, allowing for automatic refresh

```
const getToken = async (): Promise<string> => {
  const token = authService.getAccessToken();

  if (authService.isTokenExpired(token)) {
    // Token will be refreshed before returning
    return await authService.refreshToken();
  }

  return token;
};
```

Browser Support

This SDK requires modern browsers with support for:

- ES2020+ features
- `window.postMessage` API
- `HTMLIFrameElement` API

Browser	Minimum Version
Chrome	80+
Firefox	75+
Safari	13.1+
Edge	80+